

Git

NCSC101

Prof. Pranay Kumar Saha

October 8, 2025





What is Git?

- Distributed version control system
- Created by Linus Torvalds in 2005
- Free and open source
- Fast and efficient
- Most widely used version control system today



Benefits of Using Git

- Track changes over time and revert if needed
- Collaborate seamlessly with teams
- Experiment with new features without risking the main codebase
- Backup your work on remote servers
- Integrate with continuous integration/continuous deployment (CI/CD) pipelines



Installing Git

Linux (Debian/Ubuntu):

```
sudo apt-get update  
sudo apt-get install git
```

macOS:

```
brew install git
```

Windows: Download from <https://git-scm.com/download/win>



Initial Configuration

Set up your identity:

```
git config --global user.name "Your Name"  
git config --global user.email "your.email@example.com"
```

Check your configuration:

```
git config --list
```



Example: Initial Configuration Output

After setting up:

```
$ git config --list  
user.name=Your Name  
user.email=your.email@example.com  
core.repositoryformatversion=0  
...
```

This ensures commits are attributed to you correctly.



Creating a Repository

Initialize a new Git repository:

```
mkdir myproject  
cd myproject  
git init
```

This creates a hidden **.git** directory that stores all version control information.



Inside the .git Directory

The **.git** directory contains:

- **config**: Repository-specific settings
- **HEAD**: Pointer to the current branch or commit
- **index**: Information about the staging area
- **logs**: Logs of reference updates
- **objects**: All commit objects, trees, and blobs
- **refs**: References to branches and tags

Never modify these files manually!



What is the Staging Area?

Staging Area (Index): An intermediate area between working directory and repository

Purpose:

- Prepare commits carefully before finalizing
- Select specific changes to include in commit
- Review changes before committing
- Create logical, organized commits
- Separate completed work from work-in-progress

Analogy: Like a shopping cart where you review items before checkout



Why Use the Staging Area?

Benefits:

1. **Selective Commits:** Commit only related changes together
2. **Code Review:** Check staged changes with `git diff --staged`
3. **Clean History:** Create meaningful, atomic commits
4. **Flexibility:** Unstage files if you change your mind

Example: You modified 3 files but only want to commit 2 - stage only those 2 files!



Staging Area in Action

Scenario: Modified multiple files, commit selectively

```
# Check what's modified
git status

# Stage specific file
git add feature.py

# Check staged changes
git diff --staged

# Commit staged file only
git commit -m "Add new feature"

# Other modified files remain unstaged
```



Git Status

Check the status of your repository:

```
git status
```

This shows:

- **Current branch:** Which branch you're on
- **Staged changes:** Files in staging area (**green**)
- **Unstaged changes:** Modified tracked files (**red**)
- **Untracked files:** New files not in Git (**red**)



Example: Git Status

Suppose you create a new file `hello.txt`:

```
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  hello.txt

nothing added to commit but untracked files present (use "git
add" to track)
```



Adding Files to Staging Area

Add specific files:

```
git add filename.txt
```

Add all files:

```
git add .
```

Add all files with specific extension:

```
git add *.py
```



Example: After Adding Files

After `git add hello.txt`:

```
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   hello.txt
```



Committing Changes

Commit staged changes:

```
git commit -m "Your commit message"
```

Commit with detailed message:

```
git commit  
# Opens editor for detailed message
```

Stage and commit in one step:

```
git commit -am "Message for modified files"
```




Example: Commit Output

```
$ git commit -m "Initial commit"  
[main (root-commit) abc1234] Initial commit  
1 file changed, 1 insertion(+)  
create mode 100644 hello.txt
```



Viewing Commit History

View commit log:

```
git log
```

Compact view:

```
git log --oneline
```

Graphical view:

```
git log --graph --oneline --all
```



Viewing Changes

See unstaged changes:

```
git diff
```

See staged changes:

```
git diff --staged
```

Compare commits:

```
git diff commit1 commit2
```



Example: Git Diff

Suppose you modify `hello.txt`:

```
$ git diff
diff --git a/hello.txt b/hello.txt
index e69de29..b2e7fcd 100644
--- a/hello.txt
+++ b/hello.txt
@@-0,0 +1 @@
+Hello, Git!
```



What is a Branch?

- A **movable pointer** to a sequence of commits (a named line of development).
- Lets you develop features/fixes **in isolation** from **main**.
- Default branch is usually **main** (**master** in older repos).
- **Lightweight**: creating/switching branches is fast and cheap.
- **HEAD** points to your current branch; your next commit advances it.



Why & When to Branch

- **Protect main:** experiment without breaking stable code.
- **Parallel work:** multiple features and bugfixes at once.
- **Code review:** open PR/MR from a branch; run CI before merge.
- **Release lines:** maintain **release/*** or hotfix branches.
- **Disposable:** delete after merge to keep history tidy.



Good Practices

- Use clear names: **feat/short-desc**, **fix/bug-123**, **chore/ci**.
- Keep branches **small and short-lived**; open PRs early.
- Regularly **sync with main** via **rebase** (clean history) or **merge** (preserve graph).
- **Delete** branches after merging to reduce clutter.



Creating and Switching Branches

Create a new branch:

```
git branch feature-branch
```

Switch to a branch:

```
git checkout feature-branch
```

Create and switch in one command:

```
git checkout -b feature-branch
```




Managing Branches

List all branches:

```
git branch  
git branch -a # includes remote branches
```

Delete a branch:

```
git branch -d branch-name
```

Force delete:

```
git branch -D branch-name
```



Merging Branches

Merge a branch into current branch:

```
git checkout main  
git merge feature-branch
```

Fast-forward merge (if possible):

```
git merge --ff-only feature-branch
```

Create merge commit:

```
git merge --no-ff feature-branch
```

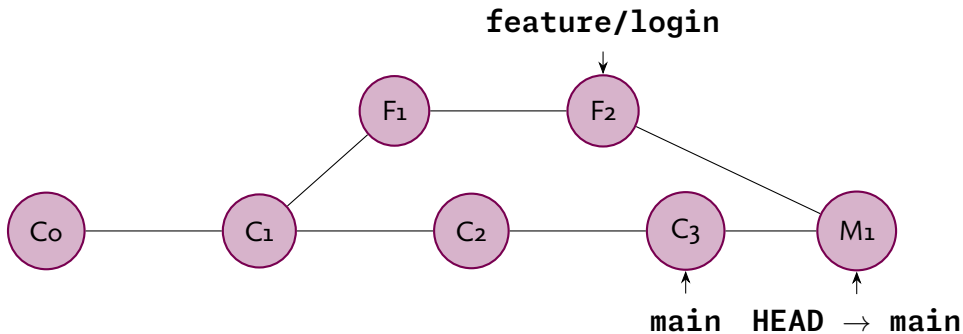


Essential Branch Commands

- List: `git branch --all`
- Create: `git branch feature/login`
- Create & switch: `git switch -c feature/login`
- Switch: `git switch feature/login`
- Update feature: `git fetch origin`
`git rebase origin/main` or
`git merge origin/main`
- Merge to main: `git switch main`
`git merge feature/login`
- Delete (merged): `git branch -d feature/login`
Force: `-D`



Merge flow



Branches are pointers to commits; merges create a new commit tying histories together.



Handling Merge Conflicts

When conflicts occur:

```
# Edit conflicted files manually
# Look for conflict markers:
<<<<<<< HEAD
your changes
=====
their changes
>>>>>>> branch-name
```

After resolving:

```
git add resolved-file.txt
git commit
```



Example: Conflict in File

Conflicted file content:

```
<<<<<< HEAD
This is the code in main branch.
=====
This is the code in feature branch.
>>>>>> feature-branch
```

Resolve by editing to the desired version.

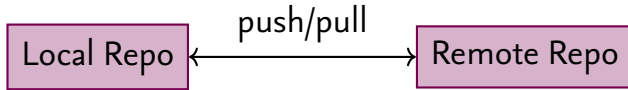


Remote Repositories

- Versions of your project hosted on the internet
- Common platforms: GitHub, GitLab, Bitbucket
- Enable collaboration with others
- Serve as backup of your code
- Usually named **origin**



Local vs Remote Diagram



Sync changes between local and remote.



Cloning a Repository

Clone a remote repository:

```
git clone https://github.com/user/repo.git
```

Clone with different directory name:

```
git clone https://github.com/user/repo.git mydir
```

Clone specific branch:

```
git clone -b branch-name https://github.com/user/repo.git
```



Adding a Remote Repository

Add a remote:

```
git remote add origin https://github.com/user/repo.git
```

View remotes:

```
git remote -v
```

Remove a remote:

```
git remote remove origin
```



Pushing to Remote

Push to remote repository:

```
git push origin main
```

Push all branches:

```
git push --all origin
```

Set upstream and push:

```
git push -u origin main  
# After this, just use: git push
```



Pulling from Remote

Fetch and merge changes:

```
git pull origin main
```

Fetch without merging:

```
git fetch origin
```

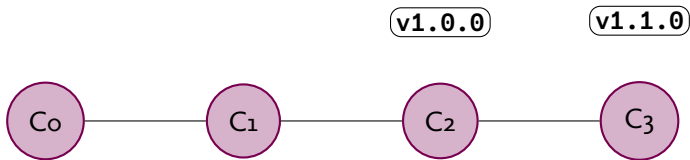
Pull with rebase:

```
git pull --rebase origin main
```



What is a Tag?

- A **fixed label** pointing at a specific object (usually a commit).
- Unlike branches, tags **do not move**.
- Types:
 - **Lightweight**: just a name pointing to a commit.
 - **Annotated** (-a): full object with message, tagger, and date.
 - **Signed** (-s): annotated tag with a cryptographic signature.
- Common use: mark releases (**v1.0.0**, **v1.1.0**).





Tagging Releases

Create a lightweight tag:

```
git tag v1.0.0
```

```
git tag -a v1.0 -m "My first release" # Annotated
```

```
git tag v1.0-lightweight # Lightweight
```

```
git tag v1.0 <commit-hash> # Tag a specific past commit
```

Push tags to remote:

```
git push origin v1.0.0
```

```
git push --tags # push all tags
```



.gitignore File

Create **.gitignore** to exclude files:

```
# Ignore compiled files
*.o
*.pyc
# Ignore directories
node_modules/
__pycache__/
# Ignore specific files
config.ini
.env
```



Commit Message Best Practices

- Write clear, concise commit messages
- Use imperative mood: "Add feature" not "Added feature"
- First line: brief summary (50 chars or less)
- Leave blank line, then detailed description
- Reference issue numbers if applicable
- Make atomic commits (one logical change per commit)



Common Branching Strategies

Git Flow:

- **main**: production-ready code
- **develop**: integration branch
- **feature/***: new features
- **release/***: release preparation
- **hotfix/***: emergency fixes

GitHub Flow:

- **main**: always deployable
- Feature branches for development
- Pull requests for code review



Typical Collaboration Workflow

```
# 1. Clone repository
git clone repo-url
# 2. Create feature branch
git checkout -b feature-name
# 3. Make changes and commit
git add .
git commit -m "Add feature"
# 4. Push to remote
git push origin feature-name
# 5. Create pull request
# (Done on GitHub/GitLab)
# 6. After merge, update main
git checkout main
git pull
```



Git Aliases

Create shortcuts for common commands:

```
git config --global alias.co checkout
git config --global alias.br branch
git config --global alias.ci commit
git config --global alias.st status
git config --global alias.unstage 'reset HEAD --'
git config --global alias.last 'log -1 HEAD'
git config --global alias.lg 'log --graph --oneline --all'
```



Resources and Further Learning

- **Official Git Documentation:** <https://git-scm.com/doc>
- **Pro Git Book:** <https://git-scm.com/book>
- **Interactive Tutorial:** <https://learngitbranching.js.org>
- **Git Cheat Sheet:** <https://education.github.com/git-cheat-sheet-education.pdf>



Git

Thank You for Listening!